

Lecture 5: ALGORITHMS - PART I

FIRST-ORDER ALGORITHMS

Phuong L. Vo

International University

September 26, 2026

Outline

- 1 Gradient descent method
- 2 Stochastic gradient descent
- 3 Subgradient method
- 4 Proximal gradient descent

- 1 Gradient descent method
- 2 Stochastic gradient descent
- 3 Subgradient method
- 4 Proximal gradient descent

Unconstrained minimization

- Unconstrained minimization problem:

$$\min_x f(x),$$

in which $x \in R^n$ f is convex and differentiable.

- Necessary and sufficient condition for a point x^* to be optimal

$$\nabla f(x^*) = 0.$$

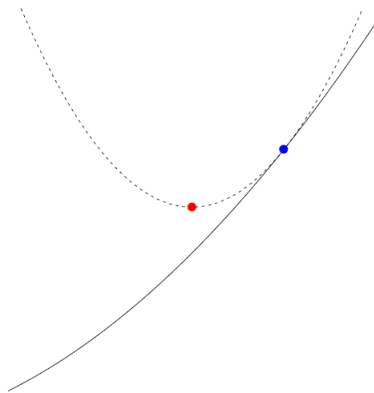
- An *iterative method* generates a sequence of points $x^{(0)}, x^{(1)}, \dots \in \mathbf{dom} f$ with $f(x^{(k)}) \rightarrow p^*$ as $k \rightarrow \infty$.
- Can be interpreted as iterative methods solve optimality condition $\nabla f(x^*) = 0$.

Gradient descent method

Choose initial point $x^{(0)}$, repeat:

$$x^{(k)} = x^{(k-1)} - t^{(k)} \nabla f(x^{(k-1)}), \quad k = 1, 2, 3, \dots$$

Gradient descent method



Blue point is x , red point is

$$x = \operatorname{argmin}_y f(x) + \nabla f(x)^T (y - x) + \frac{1}{2t} \|y - x\|_2^2$$

Choosing step-size t ?

Fixed step-size:

- can diverge if t is too big
- can be slow if t is too small

Backtracking line search:

- fix parameters $0 < \beta < 1$ and $0 < \alpha \leq 1/2$
- At each iteration, start with $t = t_{init}$, and while

$$f(x - t\nabla f(x)) > f(x) - \alpha t \|\nabla f(x)\|_2^2$$

shrink $t = \beta t$. Else, perform gradient update

$$x := x - t\nabla f(x)$$

Exact line search finding $t = \operatorname{argmin}_{s \geq 0} f(x - s\nabla f(x))$ at every iterations
→ not efficient.

Example

Minimize quadratic function $f(x) = (1/2)(x_1^2 + \gamma x_2^2)$ ($\gamma > 0$)

- with fixed step size
- with exact line search, starting at $x^{(0)} = (\gamma, 1)$,

$$x_1^{(k)} = \gamma \left(\frac{\gamma - 1}{\gamma + 1} \right)^k \quad x_2^{(k)} = \left(-\frac{\gamma - 1}{\gamma + 1} \right)^k$$

- plot x_1 vs. x_2 , error vs. k
- very slow if $\gamma \gg 1$ or $\gamma \ll 1$

Convergence rate

- For f is convex and differentiable, ∇f is Lipschitz continuous, i.e., $\|\nabla f(x) - \nabla f(y)\|_2 \leq L\|x - y\|_2$, the gradient descent with fixed step size $t \leq 1/L$ satisfies

$$f(x^{(k)}) - p^* \leq \frac{\|x^{(0)} - p^*\|_2^2}{2tk}.$$

→ convergence rate $O(1/k)$ or $O(1/\epsilon)$.

- For strongly convex f , i.e., $f(x) - \frac{m}{2}\|x\|_2^2$ is convex for some $m > 0$, and ∇f is Lipschitz continuous, the gradient descent **with fixed step size** $t \leq 2/(m + L)$ or **with backtracking line search** satisfies

$$f(x^{(k)}) - p^* \leq c^k \frac{L}{2} \|x^{(0)} - p^*\|_2^2,$$

where $c \in (0, 1)$ depends on $m, x^{(0)}$.

→ convergence rate $O(c^k)$ or $O(\log(1/\epsilon))$, exponentially fast.

Outline

- 1 Gradient descent method
- 2 Stochastic gradient descent
- 3 Subgradient method
- 4 Proximal gradient descent

Stochastic gradient descent

Consider minimizing an average of functions

$$\min_x \frac{1}{m} \sum_{i=1}^m f_i(x)$$

Since $\nabla \sum_{i=1}^m f_i(x) = \sum_{i=1}^m \nabla f_i(x)$, gradient descent repeats:

$$x^{(k)} = x^{(k-1)} - t_k \cdot \frac{1}{m} \sum_{i=1}^m \nabla f_i(x^{(k-1)}), \quad k = 1, 2, \dots$$

In comparison, stochastic gradient descent or SGD (or incremental gradient descent) repeats:

$$x^{(k)} = x^{(k-1)} - t_k \cdot \nabla f_{i_k}(x^{(k-1)}), \quad k = 1, 2, \dots$$

where $i_k \in \{1, 2, \dots, m\}$ is some chosen index at iteration k .

Two rules for choosing index i_k at iteration k :

- Randomized rule: choose $i_k \in \{1, 2, \dots, m\}$ uniformly at random
- Cyclic rule: choose $i_k = 1, 2, \dots, m, 1, 2, \dots, m, \dots$

Randomized rule is more common in practice. For randomized rule, note that

$$\mathbb{E}[\nabla f_{i_k}(x)] = \nabla f(x)$$

so we can view SGD as using an *unbiased estimate* of the gradient at each step

Main advantages of SGD:

- Iteration cost is independent of m (number of functions)
- Can also be a big savings in terms of memory usage

Example: stochastic logistic regression

Diminishing step sizes

Diminishing step sizes: choose to meet conditions:

$$\sum_{k=1}^{\infty} t_k^2 < \infty, \quad \sum_{k=1}^{\infty} t_k = \infty$$

i.e., square summable but not summable. Important here that step sizes go to zero, but not too fast. *E.g.*, $t_k = 1/k$.

Convergence rates

- Standard SGD uses diminishing step sizes and
- Recall: for convex f , **gradient descent** with **diminishing step sizes** satisfies

$$f(x^{(k)}) - f^* = O(1/\sqrt{k})$$

- When f is differentiable with **Lipschitz gradient**, gradient descent with suitable **fixed step sizes**

$$f(x^{(k)}) - f^* = O(1/k)$$

- For convex f , **SGD** with **diminishing step sizes** satisfies

$$\mathbb{E}[f(x^{(k)})] - f^* = O(1/\sqrt{k})$$

- Unfortunately this **does not improve** when we further assume f has Lipschitz gradient

Mini-batches

- Also common is mini-batch stochastic gradient descent, where we choose a random subset $B_k \subseteq \{1, \dots, m\}$, $|B_k| = b \ll m$, repeat:

$$x^{(k)} = x^{(k-1)} - t_k \cdot \frac{1}{b} \sum_{i \in I_k} \nabla f_i(x^{(k-1)}), \quad k = 1, 2, \dots$$

- Here, we are approximating full gradient by an unbiased estimate:

$$\mathbb{E} \left[\frac{1}{b} \sum_{i \in B_k} \nabla f_i(x) \right] = \nabla f(x)$$

- Using mini-batches reduces variance by a factor $1/b$, but is also b times more expensive.

Example: logistic regression

Given the problem

$$\min_{\beta} \frac{1}{n} \sum_{i=1}^n \left(-y_i x_i^T \beta + \log(1 + e^{x_i^T \beta}) \right) + \frac{\lambda}{2} \|\beta\|_2^2$$

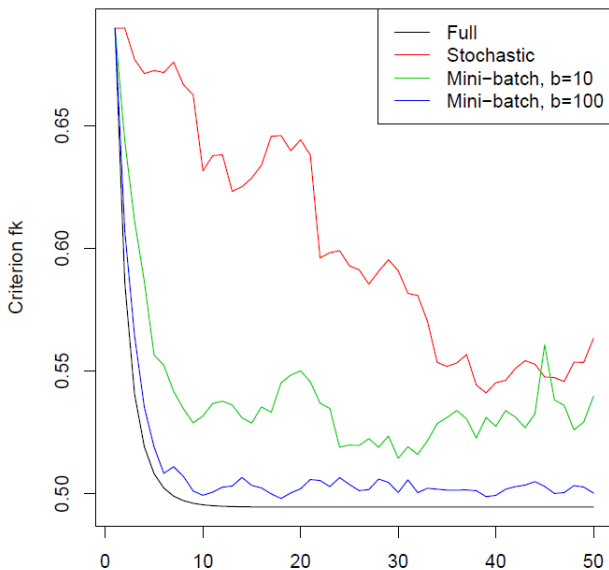
$f(\beta) = \frac{1}{n} \sum_{i=1}^n f_i(\beta)$, in which

$$f_i(\beta) = -y_i x_i^T \beta + \log(1 + e^{x_i^T \beta}) + \frac{\lambda}{2} \|\beta\|_2^2$$

Comparison between methods:

- One batch update costs $O(np)$
- One mini-batch update costs $O(bp)$
- One stochastic update costs $O(p)$

Example with $n = 10,000$, $p = 20$, all methods use fixed step sizes:



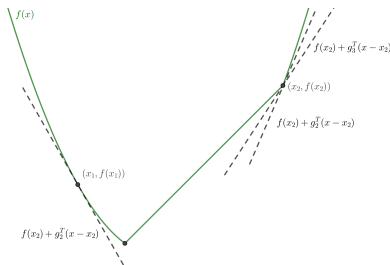
Outline

- 1 Gradient descent method
- 2 Stochastic gradient descent
- 3 Subgradient method
- 4 Proximal gradient descent

Subgradient of a function

g is subgradient of f at x if

$$f(y) \geq f(x) + g^T(y - x) \quad \forall y$$



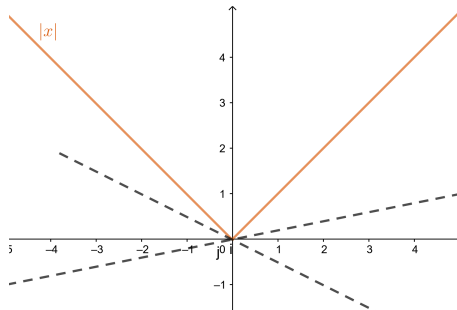
g_1 is a subgradient at x_1 ; g_2, g_3 is a subgradient at x_2 .

Subgradient of a function

- Subgradient is a global under estimator of f .
- If f is convex and differentiable, $\nabla f(x)$ is a subgradient of f at x uniquely.
- always exist

Subgradient of absolute function

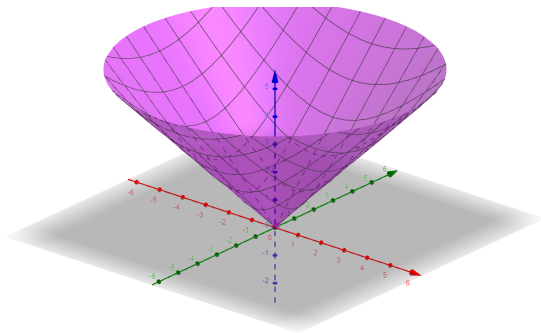
Consider $f : \mathbb{R} \rightarrow \mathbb{R}$, $f(x) = |x|$



- for $x \neq 0$, unique subgradient $g = \text{sign}(x)$
- for $x = 0$, subgradient g is any element of $[-1, 1]$

Subgradient of norm-2 function

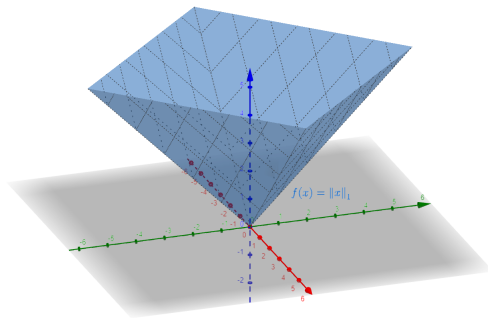
Consider $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $f(x) = \|x\|_2$



- for $x \neq 0$, unique subgradient $g = x/\|x\|_2$
- for $x = 0$, subgradient g is any element of $\{z : \|z\|_2 \leq 1\}$

Subgradient of norm-1 function

Consider $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $f(x) = \|x\|_1$



- for $x_i \neq 0$, unique i -th component subgradient $g_i = \text{sign}(x_i)$
- for $x_i = 0$, i -th component g_i is any element of $[-1, 1]$

Subgradient of max function

Consider $f(x) = \max\{f_1(x), f_2(x)\}$, for $f_1, f_2 : \mathbb{R}^n \rightarrow \mathbb{R}$ convex, differentiable

- for $f_1(x) > f_2(x)$, unique subgradient $g = \nabla f_1(x)$
- for $f_1(x) < f_2(x)$, unique subgradient $g = \nabla f_2(x)$
- for $f_1(x) = f_2(x)$, subgradient g is any point on line segment between $\nabla f_1(x)$ and $\nabla f_2(x)$

Set of all subgradients of convex f is called the subdifferential, $\partial f(x)$

- nonempty
- $\partial f(x)$ is closed and convex, even for nonconvex f
- if f is differentiable at x , then $\partial f(x) = \{\nabla f(x)\}$
- if $\partial f(x) = \{g\}$, then f is differentiable at x and $g = \{\nabla f(x)\}$

Subgradient calculus

Basic rules for convex functions:

- **Scaling:** $\partial(af) = a * \partial f$ provided $a > 0$
- **Addition:** $\partial(f_1 + f_2) = \partial f_1 + \partial f_2$
- **Affine composition:** if $g(x) = f(Ax + b)$, then

$$\partial g(x) = A^T \partial f(Ax + b)$$

- **Finite pointwise maximum:** if $f(x) = \max_{i=1,\dots,m} f_i(x)$, then

$$\partial f(x) = \text{conv}(\cup_{i:f(x)=f_i(x)} \partial f_i(x))$$

convex hull of union of subdifferentials of active function at x

Optimality condition

For any f (convex or not)

$$f(x^*) = \min_x f(x) \quad \Leftrightarrow \quad 0 \in \partial f(x^*)$$

(since $f(y) \geq f(x^*) + 0^T(y - x^*) = f(x^*)$)

Gradient method

Consider problem

$$\min_x f(x)$$

for f convex and differentiable, $\text{dom } f = \mathbb{R}^n$, **gradient descent** choose initial $x^{(0)} \in \mathbb{R}^n$, repeat:

$$x^{(k)} = x^{(k-1)} - t_k \nabla f(x^{(k-1)}), k = 1, 2, 3, \dots$$

Step size t_k chosen to be fixed and small, or by backtracking line search.
Gradient descent convergence, but the downsides are:

- Requires f differentiable — addressed this lecture
- Can be slow to converge

Subgradient method

like gradient descent, but replacing gradients with subgradients

Initialize $x^{(0)} \in \mathbb{R}^n$, repeat:

$$x^{(k)} = x^{(k-1)} - t_k g^{(k-1)}, k = 1, 2, 3, \dots$$

where $g^{(k-1)} \in \partial f(x^{(k-1)})$, any subgradient of f at $x^{(k-1)}$.

Step size choices

- Fixed step sizes: $t_k = t$
- Diminishing step sizes: choose to meet conditions:

$$\sum_{k=1}^{\infty} t_k^2 < \infty, \quad \sum_{k=1}^{\infty} t_k = \infty$$

i.e., square summable but not summable. Important here that step sizes go to zero, but not too fast. *E.g.*, $t_k = 1/k$.

Convergence rate

Assume that f is Lipschitz continuous with constant $G > 0$ and define

$$f(x_{best}^{(k)}) = \min_{i=0,\dots,k} f(x^{(i)})$$

Basic inequality after k steps

$$f(x_{best}^{(k)}) - f^* \leq \frac{R^2 + G^2 \sum_{i=1}^k t_i^2}{2 \sum_{i=1}^k t_i}$$

the convergence rate of subgradient algorithm is $O(1/\epsilon^2)$.

- For a fixed step size t , subgradient method satisfies

$$\lim_{k \rightarrow \infty} f(x_{best}^{(k)}) \leq f^* + G^2 t / 2$$

- For diminishing step sizes, subgradient method satisfies

$$\lim_{k \rightarrow \infty} f(x_{best}^{(k)}) \leq f^*$$

Example: regularized logistic regression

Given $(x_i, y_i) \in \mathbb{R}^p \times \{0, 1\}$ for $i = 1, \dots, n$, the **logistic regression** loss is

$$f(\beta) = \sum_{i=1}^n \left(-y_i x_i^T \beta + \log(1 + \exp(x_i^T \beta)) \right)$$

This is a smooth and convex, with

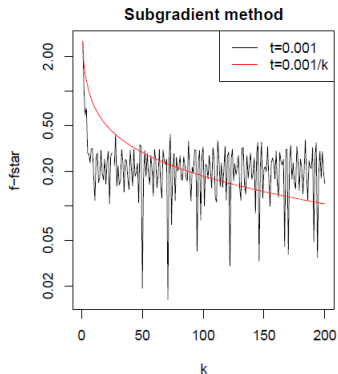
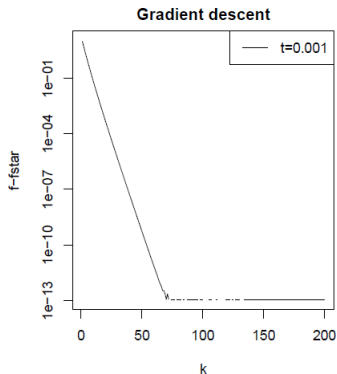
$$\nabla f(\beta) = \sum_{i=1}^n (y_i - p_i(\beta)) x_i$$

where $p_i(\beta) = \exp(x_i^T \beta) / (1 + \exp(x_i^T \beta))$, $i = 1, \dots, n$. Consider the regularized problem:

$$\min_{\beta} f(\beta) + \lambda \cdot P(\beta)$$

where $P(\beta) = \|\beta\|_2^2$, **ridge** penalty; or $P(\beta) = \|\beta\|_1$, **lasso** penalty

Ridge: use gradients; lasso: use subgradients. Example here has $n = 1000$, $p = 20$:



Step sizes hand-tuned to be favorable for each method (of course comparison is imperfect, but it reveals the convergence behaviors)

Outline

- 1 Gradient descent method
- 2 Stochastic gradient descent
- 3 Subgradient method
- 4 Proximal gradient descent

Proximal method - motivation

Consider the problem

$$\min f(x) = g(x) + h(x)$$

- g is convex, differentiable, $\text{dom } g = \mathbb{R}^n$
- f is convex, not necessarily differentiable

Recall: Gradient update for f if f differentiable:

$$x^{k+1} = x^k - t \cdot \nabla f(x^k)$$

The gradient update minimizes the *quadratic approximation* of f around x^k

$$x^{k+1} = \operatorname{argmin}_x \tilde{f}_t(x) = \operatorname{argmin}_x f(x^k) + \nabla f(x^k)^T (x - x^k) + \frac{1}{2t} \|x - x^k\|_2^2$$

Proximal method - motivation

In case f is non differentiable, but g is, we make quadratic approximation to g , leave h alone.

$$\begin{aligned}x^{k+1} &= \operatorname{argmin}_x \bar{g}_t(x) + h(x) \\&= \operatorname{argmin}_x f(x^k) + \nabla f(x^k)^T (x - x^k) + \frac{1}{2t} \|x - x^k\|_2^2 + h(x) \\&= \operatorname{argmin}_x \frac{1}{2t} \|x - (x^k - t \cdot \nabla g(x^k))\|_2^2 + h(x)\end{aligned}$$

Proximal gradient descent

Define the operator

$$\text{prox}_{h,t}(y) = \operatorname{argmin}_x \frac{1}{2t} \|x - y\|_2^2 + h(x)$$

Among all possible x , find one that makes $h(x)$ small, but does not move too far away from the point y .

Proximal gradient descent: choose initial $x^{(0)}$, repeat:

$$x^{(k+1)} = \text{prox}_{h,t_k}(x^{(k)} - t_k \cdot \nabla g(x^{(k)}))$$

Equivalently, in each iteration:

- Gradient descent: $y^{(k)} = x^{(k)} - t_k \cdot \nabla g(x^{(k)})$
- Proximal: $x^{(k+1)} = \operatorname{argmin}_x \frac{1}{2t_k} \|x - y^{(k)}\|_2^2 + h(x)$

What good did this do?

It looks like we just swapped one minimization problem for another!!!

- $\text{prox}_{h,t}(\cdot)$ has a closed-form for many important functions h
- Mapping $\text{prox}_{h,t}(\cdot)$ doesn't depend on g at all, only on h
- Smooth part g can be complicated, we only need to compute its gradients
- Can decompose a large-scale problem

Convergence analysis

Given $f(x) = g(x) + h(x)$, we assume

- $g : \mathbb{R}^n \rightarrow \mathbb{R}$ is convex, differentiable, and ∇g is Lipschitz continuous with constant $L > 0$ ($\|g(x) - g(y)\| \leq L\|x - y\|$, for all $x, y \in \mathbb{R}^n$).
- h is convex, and $\text{prox}_t(x) = \arg\min_z \{\frac{1}{2t}\|x - z\|_2^2 + h(z)\}$ can be evaluated

Theorem: *Proximal gradient descent with fixed step size $t \leq 1/L$ satisfies*

$$f(x^{(k)}) - f^* \leq \frac{\|x^{(0)} - x^*\|_2^2}{2tk}$$

and same result holds for backtracking, with t replaced by β/L

Proximal gradient descent has convergence rate $O(1/k)$ or $O(1/\epsilon)$. Same as gradient descent!

Iterative Shrinkage-Thresholding Algorithm for LASSO

Given labels $d = \begin{bmatrix} d_1 \\ d_2 \\ \dots \\ d_n \end{bmatrix} \in \mathbb{R}^n$, and feature matrix $A = \begin{bmatrix} a_1^T \\ a_2^T \\ \dots \\ a_n^T \end{bmatrix} \in \mathbb{R}^{n \times p}$,

in which feature vector $a_i = \begin{bmatrix} a_{i1} \\ a_{i2} \\ \dots \\ a_{ip} \end{bmatrix} \in \mathbb{R}^p$ corresponds to label d_i .

The LASSO problem is given by

$$\min_w f(w) = g(w) + h(w) = \frac{1}{2} \|Aw - d\|_2^2 + \lambda \|w\|_1$$

Proximal method:

① **Gradient descent:** $v^{(k)} = w^{(k)} - tA^T(Aw^{(k)} - d)$

② **Proximal mapping:**

$$w^{(k+1)} = \text{Prox}_t(v^{(k)}) = \underset{w}{\operatorname{argmin}} \frac{1}{2t} \|w - v^{(k)}\|_2^2 + \lambda \|w\|_1$$

Iterative Shrinkage-Thresholding Algorithm for LASSO

Proximal mapping can be computed analytically

$$\begin{aligned} & \operatorname{argmin}_w \frac{1}{2t} \|w - v^{(k)}\|_2^2 + \lambda \|w\|_1 \\ &= \sum_{i=1}^p \operatorname{argmin}_{w_i} \frac{1}{2} (w_i - v_i^{(k)})^2 + \lambda t |w_i|, i = 1, \dots, p \end{aligned}$$

Since $\partial|w_i| = \begin{cases} 1, & \text{if } w_i > 0, \\ -1, & \text{if } w_i < 0, \\ [-1, 1], & \text{if } w_i = 0, \end{cases}$ one solution is the soft-threshold operator

$$w_i^{(k+1)} = z_i^* = \begin{cases} v_i^{(k)} - \lambda t & \text{if } v_i^{(k)} > \lambda t \\ 0 & \text{if } -\lambda t \leq v_i^{(k)} \leq \lambda t \\ v_i^{(k)} + \lambda t & \text{if } v_i^{(k)} < -\lambda t \end{cases}$$

Projected gradient descent (PGD)

Given closed, convex set $C \in \mathbb{R}^n$,

$$\min_{x \in C} g(x) \iff \min_x g(x) + I_C(x),$$

where I_C is the indicator function of C , $I_C(x) = \begin{cases} 0 & \text{if } x \in C \\ \infty & \text{if } x \notin C \end{cases}$.

$$\begin{aligned} \text{prox}_{I_C}(x) &= \operatorname{argmin}_z \frac{1}{2} \|x - z\|_2^2 + I_C(z) \\ &= \operatorname{argmin}_{z \in C} \frac{1}{2} \|x - z\|_2^2 \\ &= P_C(x), \end{aligned}$$

projection operator on to C .

Projected gradient descent

Therefore proximal gradient step is

$$x^{(k+1)} = P_C(x^{(k)} - t\nabla g(x))$$

i.e., perform usual gradient update and then project back onto C .

References

- Convex optimization book by Stephen Boyd
- Convex Optimization course at CMU